



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algorithms and Data Structures I.

Gábor Pusztai

Department of Algorithms And Their Applications
Faculty of Informatics
ELTE - Eötvös Loránd University, Budapest

April 13, 2026

Content in this Practice I

- 1 Binary search tree (BST)
 - Building a Binary Search Tree
 - Tree Reconstruction

- 2 Operations
 - Search
 - Insertion
 - Deletion
 - "Successor"

Binary search tree (BST)

Binary search tree definition

A binary tree is called a *search tree* if the following requirements are true for every internal node and its key y :

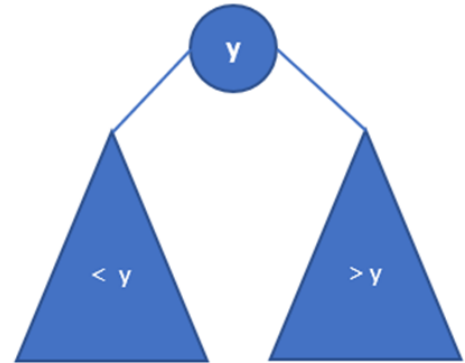
- In the node's left subtree, for any node's key x , $x < y$.
- In the node's right subtree, for any node's key z , $z > y$.

A binary tree is called a *sorting tree* if the following requirements are true for every internal node and its key y :

- In the node's left subtree, for any node's key x , $x \leq y$.
- In the node's right subtree, for any node's key z , $z \geq y$.

Binary search tree property

The following condition is true for an arbitrary internal node of the tree.



What is the error in the following definition?

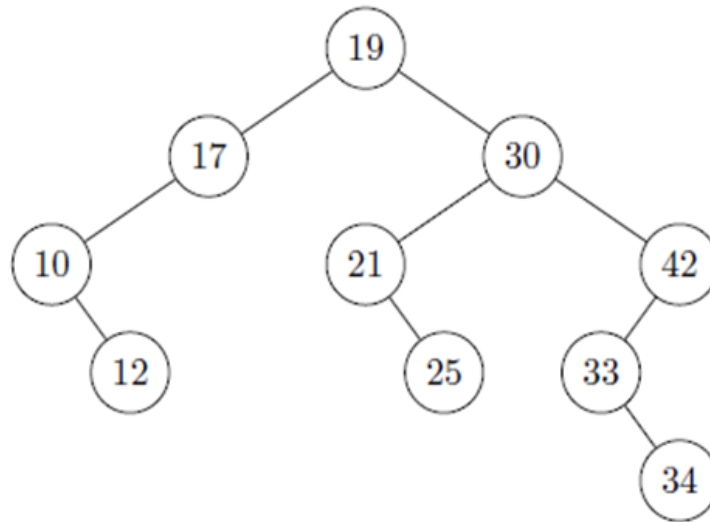
- „The key of every internal node's left child is smaller, and the key of its right child is larger.”
- **Error:** It is not enough to state this for the node's children; it must hold for **all** keys in the node's left and right subtrees!

Task: Building a Binary Search Tree

- Build a binary search tree from the following elements by inserting keys one after another:
- 19, 17, 30, 10, 21, 12, 42, 25, 33, 34

Solution: Building a Binary Search Tree

- Build a binary search tree from the following elements by inserting keys one after another:
- 19, 17, 30, 10, 21, 12, 42, 25, 33, 34



Task: Tree Reconstruction

- We can reconstruct the shape of the binary search tree from preorder or postorder traversals.

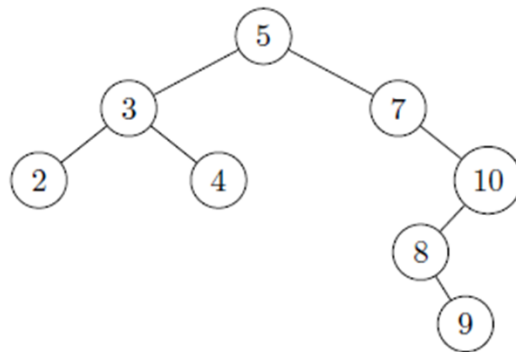
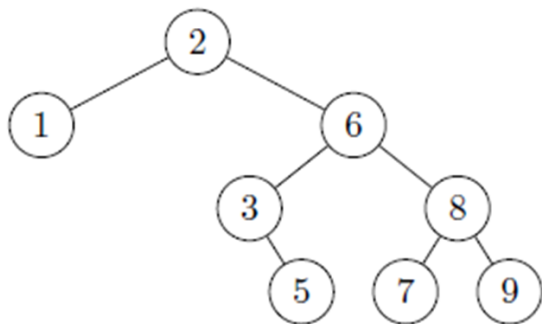
Preorder: 2, 1, 6, 3, 5, 8, 7, 9 **Postorder:** 2, 4, 3, 9, 8, 10, 7, 5

Solution: Tree Reconstruction

- We can reconstruct the shape of the binary search tree from preorder or postorder traversals.

Preorder: 2, 1, 6, 3, 5, 8, 7, 9

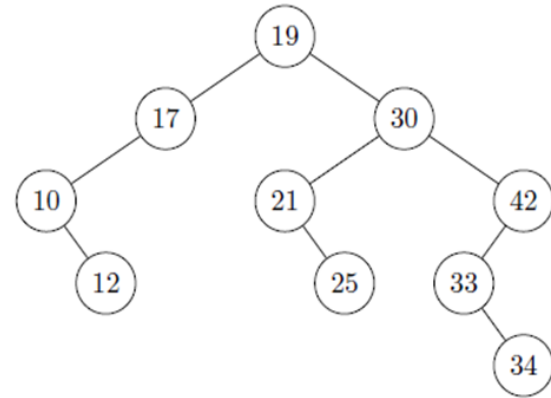
Postorder: 2, 4, 3, 9, 8, 10, 7, 5



Operations

Questions for the constructed tree

- Find key 25!
- Find key 23!
- Which keys would fit in the left subtree of 21? (20)
- Which keys would fit in the right subtree of 12? (13..16)
- Which traversal gives the keys in ascending order of magnitude?
(**Inorder**)

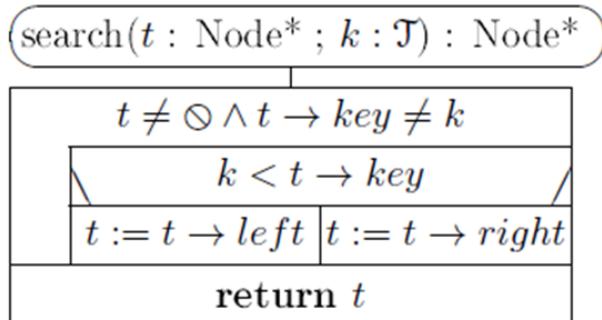


Basic operations of binary search trees

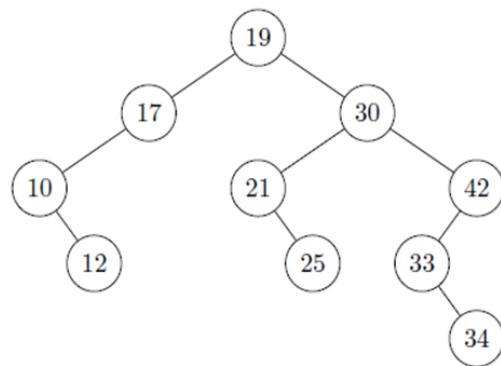
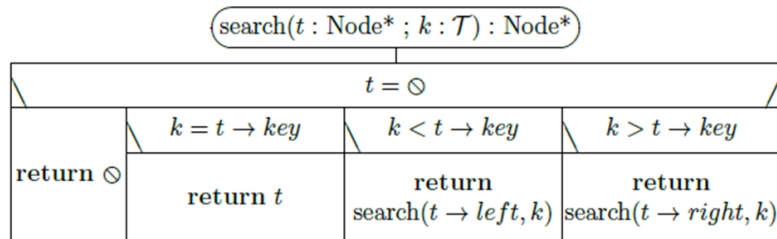
- We can create recursive or iterative versions for the operations.
- **Operations:**
 - `search(t, k)`: searching for an element with key `k`.
 - `insert(t, k)`: inserting an element with key `k`.
 - `min(t)`: address of the node with the minimum key.
 - `remMin(t, minp)`: removing the minimum element.
 - `del(t, k)`: deleting the element with key `k`.
- Every operation has a cost of $O(h)$, where h is the height of the tree.

Search operation (Iterative and Recursive)

Iterative:



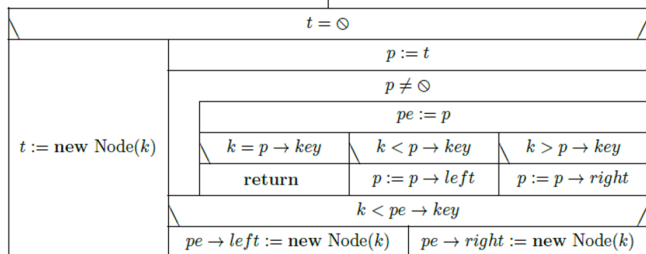
Recursive:



Insertion operation (Iterative and Recursive)

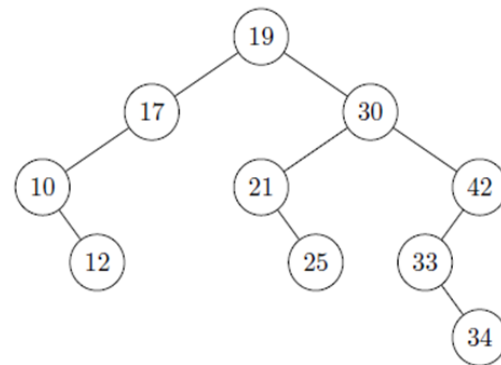
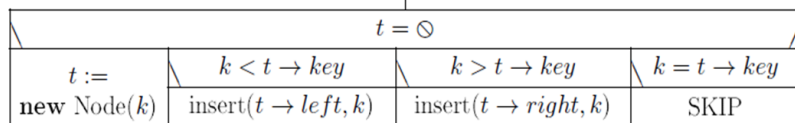
Iterative:

$\text{insert}(\&t : \text{Node}^* ; k : \mathcal{T})$



Recursive:

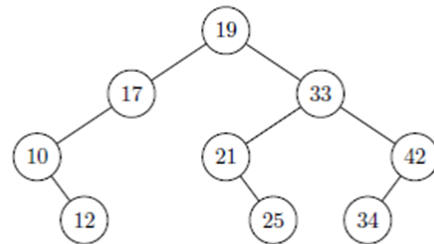
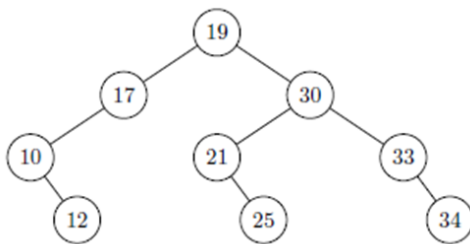
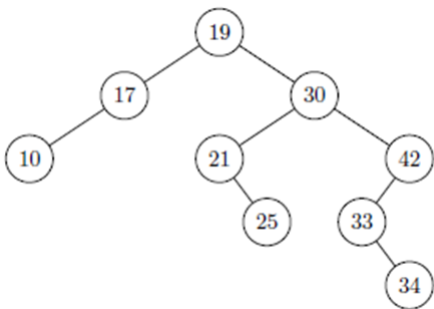
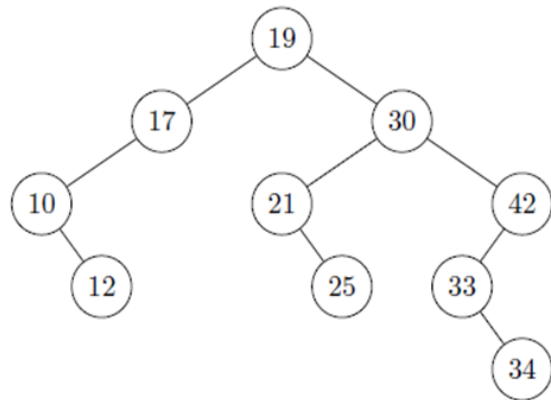
$\text{insert}(\&t : \text{Node}^* ; k : \mathcal{T})$



Cases of Deletion Operation

- **Deleting a leaf:** e.g., 12
- **Deleting an internal node with one child:** e.g., 42
- **Deleting an internal node with two children:** e.g., 30 (the smallest element from the right subtree replaces it)

Original tree:



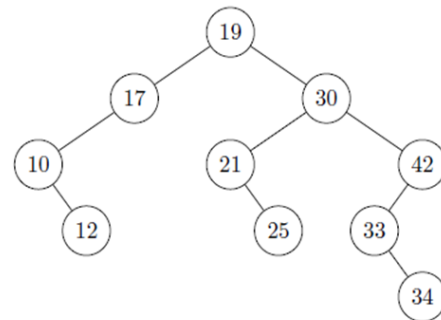
Deletion operation

$\text{del}(\&t : \text{Node}^* ; k : \mathcal{T})$

$t \neq \ominus$			
$k < t \rightarrow \text{key}$	$k > t \rightarrow \text{key}$	$k = t \rightarrow \text{key}$	SKIP
$\text{del}(t \rightarrow \text{left}, k)$	$\text{del}(t \rightarrow \text{right}, k)$	$\text{delRoot}(t)$	

$\text{delRoot}(\&t : \text{Node}^*)$

$p := t$		
$t \rightarrow left = \ominus$	$t \rightarrow right = \ominus$	$t \rightarrow left \neq \ominus \wedge t \rightarrow right \neq \ominus$
$t := p \rightarrow right$	$t := p \rightarrow left$	$\text{remMin}(t \rightarrow right, q)$
		$q \rightarrow left := p \rightarrow left$
		$q \rightarrow right := p \rightarrow right$
		$t := q$
delete p		

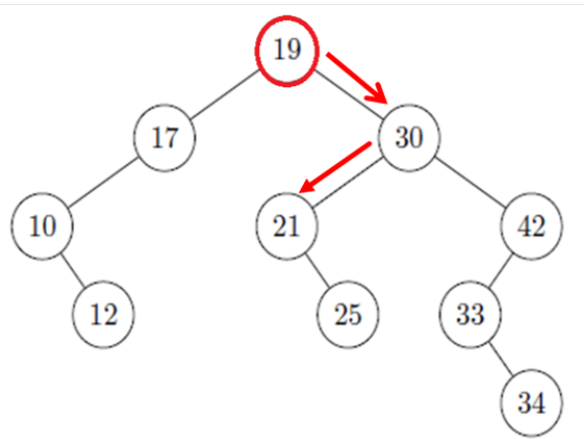


$\text{remMin}(\&t, \&\text{minp} : \text{Node}^*)$

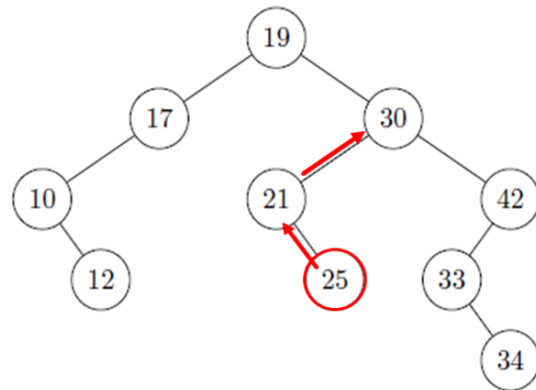
$t \rightarrow \text{left} = \ominus$	
$\text{minp} := t$	$\text{remMin}(t \rightarrow \text{left}, \text{minp})$
$t := \text{minp} \rightarrow \text{right}$	
$\text{minp} \rightarrow \text{right} := \ominus$	

The "successor" operation - $inorder_next(p)$

- We are looking for the successor of the current element according to the inorder traversal.
- The parent pointer is also needed for the operation!



- If it has a right subtree: its minimum.



- If not: the first ancestor in whose left subtree we are located.

The "successor" operation - $inorder_next(p)$

$inorder_next(p : Node3^*) : Node3^*$

$q := p \rightarrow right$

$q \neq \emptyset$

$q \rightarrow left \neq \emptyset$

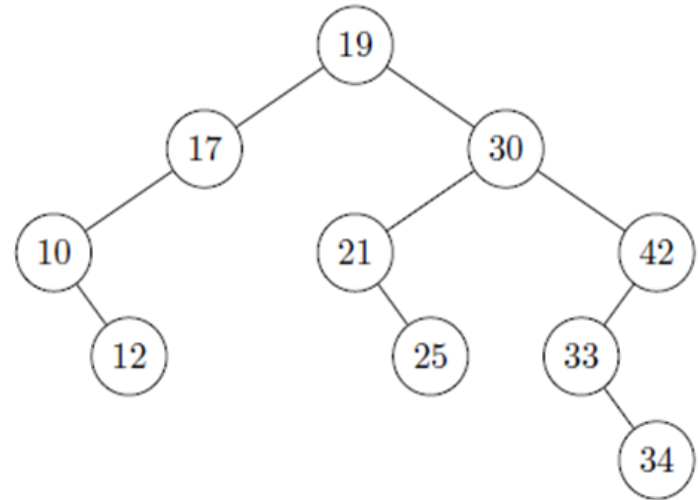
$q := p \rightarrow parent$

$q := q \rightarrow left$

$q \neq \emptyset \wedge q \rightarrow left \neq p$

$p := q ; q := q \rightarrow parent$

return q



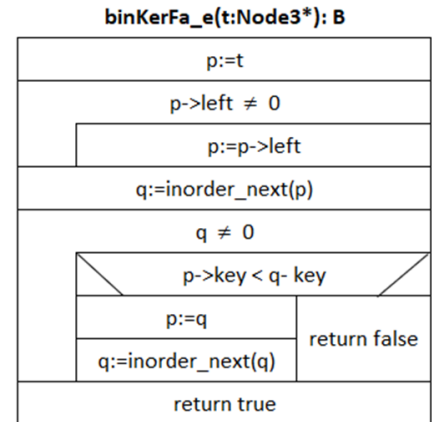
Checking binary search tree property

- **Task:** given a binary tree in linked representation, check if it is a binary search tree. The nodes of the tree are of type *Node3*, meaning a parent pointer is available. Assume the tree is not empty.

Checking binary search tree property

- **Task:** given a binary tree in linked representation, check if it is a binary search tree. The nodes of the tree are of type *Node3*, meaning a parent pointer is available. Assume the tree is not empty.
- Imagine an iterative algorithm that traverses the tree using the newly learned *inorder_next(p)* algorithm, while checking if the order of keys is strictly monotonically increasing.

- **Idea:** Check the strictly monotonically increasing order with inorder traversal.



Thank you for your attention!